INSTITUTO TÉCNICO RICALDONE · ESPECIALIDAD DE DESARROLLO DE SOFTWARE

Repaso para la Defensa Externa de PetCore

Sistema de gestión veterinaria: sistema web (HTML, CSS, Bootstrap, JavaScript sin frameworks), API (Java, Spring Boot, JPA/Hibernate), base de datos Oracle Cloud y app móvil Android/iOS con Capacitor.

Frontend → Vercel (petcore-seven.vercel.app)

API / Backend → Heroku (petcore-8afada45fabc.herokuapp.com)

Base de datos → Oracle Cloud, gestionada con SQL Developer

App móvil → Capacitor (Android / iOS)

Contenido

1. Rúbrica oficial de evaluación externa
2. Arquitectura del Backend (API Java / Spring Boot)
3. Arquitectura del Frontend y la App Móvil
4. Base de datos (Oracle Cloud)
5. Roles, autenticación y funcionalidad CRUD
6. Conceptos clave explicados
7. Banco de preguntas para la defensa externa
8. Guion de presentación

SECCIÓN 1

Rúbrica oficial de evaluación externa

Estos son los 10 criterios reales que evalúa el jurado. Úsalos como checklist de preparación del equipo.

- [ ] **1. Documentación del sistema.** Manual de usuario con imágenes que muestre cómo se realizan los procesos + documentación técnica para poner en marcha el software + diagramas que expliquen el funcionamiento.
- [ ] **2. Diseño de las interfaces web.** Todas las interfaces ajustan su disposición según el tamaño de pantalla; fácil interacción, comprensibles, con diseño estético y profesional.
- [ ] **3. Diseño de las interfaces móviles.** Mismo criterio que el anterior, aplicado a la app móvil.
- [ ] **4. Funcionalidad del sistema web.** Todos los mantenimientos (CRUD) funcionan correctamente y están validados en el Frontend y en el Backend.
- [ ] **5. Funcionalidad de la aplicación móvil.** Igual que el criterio 4, pero en la app móvil.
- [ ] **6. Validaciones e integridad de los datos.** Se valida que los datos correspondan al tipo de información solicitado (campos numéricos, fechas, textos, imágenes) tanto en el sistema web como en la app móvil, verificado por la API.
- [ ] **7. Autenticación y roles de usuario.** El sistema tiene inicio de sesión y demuestra claramente el manejo de roles a nivel de Frontend y Backend, y las funciones que cada uno puede realizar.
- [ ] **8. Protección de endpoints y vistas.** El diagrama de casos de uso evidencia la implementación de autenticación/autorización; demostración desde el frontend de endpoints públicos y protegidos; se bloquea el botón "atrás" del navegador tras cerrar sesión.
- [ ] **9. Despliegue del Backend, sistema web y móvil.** La API está desplegada y configurada para recibir peticiones de orígenes definidos (CORS); el sistema web es accesible por una dirección pública; la app móvil está instalada en un dispositivo físico Android.
- [ ] **10. Dominio.** El equipo responde con seguridad, demuestra dominio de la temática, usa lenguaje técnico y descripciones acertadas del trabajo realizado.

> Orientación oficial del jurado: Frontend en HTML/CSS/Bootstrap/JavaScript sin frameworks, desplegado en Vercel; Backend en Java + Spring Boot + JPA/Hibernate, desplegado en Heroku; base de datos en Oracle Cloud gestionada con SQL Developer.

SECCIÓN 2 · BACKEND · JAVA + SPRING BOOT

Arquitectura de la API de PetCore

Monolito modular en Java 17 con Spring Boot 4.1, JPA/Hibernate sobre Oracle, desplegado en Heroku. Paquete raíz `PTC_PETCORE.API`.

Origen del proyecto (Spring Initializr)

El `pom.xml` hereda de `spring-boot-starter-parent` (v4.1.0), lo que confirma que el proyecto nació desde **start.spring.io** (Spring Initializr), con Maven como gestor de dependencias y Java 17. El equipo fue agregando dependencias por fases, documentado en sus propios comentarios del pom ("1. OBLIGATORIA", "3. HABITUALES: agregar solo si estás creando el proyecto desde cero").

- **Group:** PTC_PETCORE · **Artifact:** API · **Packaging:** Jar
- **Java:** 17 (fijado también en `system.properties` para Heroku)
- **Build tool:** Maven (`pom.xml`, wrapper `mvnw` / `mvnw.cmd`)
- Incluye `ServletInitializer.java` (extiende `SpringBootServletInitializer`): permite empacar también como WAR, aunque se despliega como Jar ejecutable en Heroku.

Dependencias de la API y para qué sirve cada una

DEPENDENCIA	PARA QUÉ SE USA EN PETCORE
`spring-boot-starter-data-jpa`	JPA/Hibernate: mapea las 48 tablas Oracle a entidades Java y genera los repositorios.
`spring-boot-starter-web/-webmvc`	Spring MVC: controllers REST, serialización JSON automática.
`spring-boot-starter-security`	Autenticación/autorización: filtra cada request, aplica roles.
`spring-security-crypto`	Utilidades criptográficas usadas junto al hashing Argon2 de contraseñas.
`jjwt-api / impl / jackson`	Firma y valida los JWT guardados en la cookie `authToken`.
`spring-boot-starter-validation`	Bean Validation (`@NotBlank`, `@Email`, `@Pattern`...) sobre los DTO de entrada.
`spring-boot-starter-websocket`	Soporte WebSocket de bajo nivel para el endpoint `/ws`.
`spring-boot-starter-mail`	Envío de correos: código de recuperación de contraseña.
`google-api-client`	Verifica el ID Token de "Iniciar sesión con Google" del lado del servidor.
`cloudinary-http44`	Sube y gestiona imágenes (fotos de perfil, mascotas).
`bcprov-jdk18on` (BouncyCastle)	Cifrado ECDH/VAPID de Web Push.
`web-push`	Notificaciones push al navegador (Web Push API + VAPID).
`firebase-admin`	Push nativo a la app móvil vía Firebase Cloud Messaging.
`ojdbc11`	Driver JDBC oficial de Oracle.
`lombok`	Genera getters/setters/toString por anotación.

DEPENDENCIA	PARA QUÉ SE USA EN PETCORE
`dotenv-java`	Carga variables de entorno desde `.env` en desarrollo local.
`spring-boot-starter-tomcat`	Servidor embebido que ejecuta la API.

Arquitectura modular por feature (no por capa)

En vez de carpetas `controllers/`, `services/` gigantes mezcladas, PetCore usa **~47 módulos verticales** (uno por entidad/tabla) bajo `modules/`. Cada módulo repite siempre 6 carpetas:

- **controller/** — clases `@RestController`, reciben el HTTP request y devuelven `ResponseEntity<ApiResponse<T>>`.
- **service/** — lógica de negocio; nunca usa try/catch, lanza excepciones propias del módulo.
- **repository/** — interfaces `JpaRepository<Entidad, Long>`.
- **entity/** — clases `@Entity` mapeadas a las tablas `TBL_*`, con Lombok y relaciones JPA.
- **dto/** — objetos de transferencia con Bean Validation.
- **exceptions/** — 5 excepciones propias por módulo heredando de clases globales.

Paquetes transversales

PAQUETE	RESPONSABILIDAD
`security/config`	`SecurityConfig.java` (1219 líneas) — toda la matriz de autorización por rol y endpoint.
`security/cors`	`CorsConfig.java` — orígenes, métodos y headers permitidos.
`security/jwt`	`JWTUtils`, `JwtCookieAuthFilter`, `AuthenticatedPrincipal`, `RestAuthHandlers`.
`exceptions/global`	`GlobalExceptionHandler`: traduce cualquier excepción a una respuesta JSON consistente.
`config/websocket`	`WebSocketConfig` + `EventoWebSocketHandler`: endpoint `/ws`.
`config/cloudinary`	Credenciales y cliente de subida de imágenes.
`config/crypto`	`Argon2PasswordConfig`: algoritmo de hashing de contraseñas.
`config/paginacion`	Helper para las búsquedas paginadas.
`config/async`	Ejecuta tareas (ej. notificaciones) sin bloquear el hilo HTTP.
`response`	`ApiResponse<T>`: envoltorio estándar `{success, message, data}` de todas las respuestas.

> El módulo `auth/` es especial, no sigue el patrón CRUD: contiene `AuthController`, `ContraseniaController`, `RecuperacionController`, el verificador de Google y la recuperación de contraseña por correo.

CORS: ¿quién puede llamar a la API?

Definido en `security/cors/CorsConfig.java`, un único bean `CorsConfigurationSource` registrado para `/**`.

- **Orígenes permitidos:** localhost/127.0.0.1 (desarrollo), `*.herokuapp.com`, `petcore-seven.vercel.app` y `petcore-oerh.vercel.app`, `localhost:5173` (Vite), `https://localhost` y `capacitor://localhost` (apps nativas), más orígenes extra vía variable de entorno `security.cors.extra-origins`.

- **allowCredentials = true** — obligatorio porque la sesión viaja en cookie, no en header.
- **Métodos:** GET, POST, PUT, PATCH, DELETE, OPTIONS.
- **Headers permitidos:** Origin, Content-Type, Accept, Authorization, X-Requested-With, Cookie...
- **Headers expuestos:** Set-Cookie, Cookie, Authorization, Content-Disposition.
- **maxAge:** 3600s.

> **Detalle fino:** `WebSocketConfig.java` repite manualmente la misma lista de orígenes para el handshake de `/ws`, porque Spring Security no comparte automáticamente la configuración de CORS HTTP con WebSocket.

Spring Security: autenticación, roles y "deny by default"

`security/config/SecurityConfig.java` (1219 líneas) — el archivo más importante de la API.

- CSRF deshabilitado (API stateless con JWT, sin formularios HTML tradicionales).
- `sessionCreationPolicy(STATELESS)` — no hay sesión guardada en servidor.
- `RestAuthHandlers` como `authenticationEntryPoint` y `accessDeniedHandler` → respuestas 401/403 en JSON consistente.
- `JwtCookieAuthFilter` insertado antes de `UsernamePasswordAuthenticationFilter`.

Roles del sistema: `ROLE_Administrador` `ROLE_Veterinario` `ROLE_Recepcionista` `ROLE_Farmaceutico` y **ROLE_Propietario** (dueño de mascotas, app móvil). Constante `STAFF = {ADMIN, VET, FARM, RECEP}` y helper `staffMas(extra...)`.

El patrón clave: "deny by default"

Cada tabla tiene su método `reglasX(auth)` con reglas explícitas por endpoint. Al final de toda la cadena:

```
auth.anyRequest().denyAll();
```

Cualquier ruta nueva sin regla explícita queda bloqueada automáticamente — es el punto de seguridad más fuerte para mencionar en la defensa.

Ejemplos reales de reglas por endpoint

MÉTODO	RUTA	QUIÉN ACCEDE
GET	`/api/cfgs_sistema`	Público (pinta el login antes de autenticar)
POST	`/api/empleados/primer-administrador`	Público — solo mientras no hay ningún empleado (bootstrap)
GET	`/api/push/vapid-public-key`	Público — clave VAPID necesaria antes del login
POST	`/api/auth/login/empleado`	Público (es el propio login)
GET	`/api/alergia_mascota/por_mascota/{id}`	Veterinario y Propietario
POST	`/api/citas`	Recepcionista y Propietario
PATCH	`/api/citas/{id}/estado`	Veterinario y Recepcionista
GET	`/api/empleados/**`	Solo Administrador

MÉTODO	RUTA	QUIÉN ACCEDE
—	`/ws/**`	Público (handshake abierto, mejora pendiente)

> Debilidad honesta a mencionar si preguntan: el handshake de `/ws/**` está en `permitAll()` a propósito — el canal WebSocket aún no valida el JWT en la conexión inicial.

JWT firmado + cookie httpOnly

`security/jwt/JWTUtils.java`: firma con **HMAC-SHA256**, secreto en variable de entorno `SECRET_KEY_JWT`. Claims: `id`, `rol`, `tipo` ("EMPLEADO"/"PROPIETARIO"), `sub`=correo, `iss`, `exp`.

`security/jwt/JwtCookieAuthFilter.java`: filtro `OncePerRequestFilter`, lee la cookie `authToken` (empleados) o `authTokenPropietario` (dueños). **Sesión deslizante:** cada request válido renueva la cookie (TTL 900s = 15 min).

Dos audiencias, dos cookies: **empleados** (portal web) y **propietarios** (app móvil), cada uno con su propio login y endpoint `/api/auth/me/...`, firmados con la misma utilidad pero mantenidos separados.

Entidades JPA y relaciones principales

- **EmpleadosEntity** (`TBL_EMPLEADOS`): "hub" del sistema. `@ManyToOne` a Roles (obligatorio) y Turnos (opcional). ~17 relaciones `@OneToMany` inversas (Expedientes, Citas, Ventas, Hospitalización, etc.).
- **RolesEntity** (`TBL_ROLES`): catálogo simple de los 5 roles.
- **PropietariosEntity** (`TBL_PROPIETARIOS`): usuarios del portal móvil, incluye `pro_google_id`.
- **MascotasEntity** (`TBL_MASCOTAS`): `@ManyToOne` a Raza, Pelaje y Propietario; `@OneToMany` a Citas, Cartillas, Expediente, Hospitalización, etc.

> Convención técnica: relaciones directas con `fetch=LAZY`, PKs con `GenerationType.IDENTITY` (compatible con Oracle), y `hibernate.default_batch_fetch_size=50` para evitar el problema N+1.

Validaciones e integridad de datos

Bean Validation en los DTO, ejemplo real de `EmpleadosDTO`:

```
@NotBlank
@Email
private String empCorreo;

@Pattern(regexp = "^[0-9]{8}-[0-9]$")
private String empDui;

@Past
private LocalDate empFechaNac;

@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String empContrasenia;
```

El campo de contraseña nunca se devuelve en las respuestas JSON gracias a `WRITE_ONLY`.

`exceptions/global/GlobalExceptionHandler.java` centraliza el manejo de errores: `MethodArgumentNotValidException` → 400, `NotFoundException` → 404, `DuplicateException` / `DataIntegrityViolationException` → 409, `BadCredentialException` → 401, `CuentaInactivaException` → 403, `Exception` genérico → 500.

WebSockets en el Backend

WebSocket **puro/crudo** (sin STOMP, sin broker de mensajes). `config/websocket/WebSocketConfig.java` registra el endpoint `/ws`; `EventoWebSocketHandler.java` mantiene todas las sesiones conectadas y expone `difundir(evento, datos)` que hace broadcast total a todas las sesiones.

- Ping cada 25s (evita que Heroku corte conexiones inactivas a los 55s).
- Pool de hilos dedicado para que un broadcast lento no bloquee el hilo HTTP.
- Uso: campanita de notificaciones, indicador conectado/desconectado, actualización en caliente de citas/asistencias/configuración.

Configuración y despliegue en Heroku

Todo externalizado a variables de entorno: BD (`DB_URL`, `DB_USER`, `DB_PASSWORD`...), JWT (`SECRET_KEY_JWT`, `JWT_EXPIRATION`), cookies (`COOKIE_SECURE`, `COOKIE_SAMESITE`, `SESSION_TTL_SEGUNDOS`), Cloudinary, VAPID, Firebase, SMTP, Google OAuth. Pool HikariCP máx. 8 conexiones, Tomcat máx. 30 hilos (ajustado al plan gratuito).

```
web: java -Dserver.port=$PORT $JAVA_OPTS -jar target/API-0.0.1-SNAPSHOT.jar
```

Dominio real: `petcore-8afada45fabc.herokuapp.com`. Un workflow de GitHub Actions (`keep-alive.yml`) hace ping periódico para reducir el "cold start" del dyno gratuito.

SECCIÓN 3 · FRONTEND · HTML/CSS/JS PURO + APP CAPACITOR

Arquitectura del Frontend y la App Móvil

HTML, CSS, Bootstrap 5.3.8 y JavaScript sin frameworks, desplegado en Vercel. La app móvil usa Capacitor para empaquetar ese mismo frontend dentro de un WebView nativo Android/iOS.

Patrón Controller / Service, páginas por rol

- **js/controllers/** (~69 archivos) — un controller por página, maneja eventos y pinta datos en el DOM.
- **js/services/** (~136 archivos) — un service por entidad; son los únicos que hacen `fetch` a la API.
- **js/utils/** — helpers compartidos: alertas (SweetAlert2), fechas, selects/datepickers, lector de código de barras, generación de PDF.

pages/ organizado por rol

- **Administrador** 20 páginas: personal, inventario (+categorías/marcas/proveedores), compras, citas, servicios, turnos, reportes, configuración...
- **Veterinario** 16 páginas: expedientes clínicos, citas, cuidados, alergias, hospitalización, posología, reportes...
- **Recepcionista** 9 páginas: citas, sala de espera, registro de mascotas/propietarios, catálogos...
- **Farmacéutico** 7 páginas: punto de venta, stock, posología, registro de ventas...

Más `pages/Cuentas/` (login, recuperar contraseña) y `pages/Primer_Inicio/` (asistente de configuración inicial y creación del primer administrador).

Autenticación: sesión por cookie, no por localStorage

Dato clave: el proyecto migró de JWT-en-localStorage a cookie httpOnly. El comentario en `login_controller.js` dice textualmente: *"ya no se guarda nada en localStorage/sessionStorage aquí"*.

1. **Login:** `login_controller.js` → `AuthModule.iniciarSesion()` llama a `login_service.js`, que hace `POST /api/auth/login/empleado`.
2. **Cookie httpOnly:** el backend pone la cookie `authToken`; JavaScript nunca puede leerla ni manipularla.
3. **credentials: "include":** `fetch_con_credenciales.js` parchea `window.fetch` globalmente para mandar siempre las cookies.
4. **Verificación de sesión:** `sesion_service.js` → `verificarSesionSync()` hace un XHR síncrono contra `GET /api/auth/me/empleado` para bloquear la carga de la página hasta saber si hay sesión válida.
5. **Logout:** `POST /api/auth/logout` (el servidor borra la cookie); el frontend también limpia `sessionStorage` y el Cache Storage.

> ¿Qué SÍ hay en localStorage? Solo preferencias de interfaz sin información sensible: tema claro/oscuro, animaciones, opacidad de fondo, formato de hora, si ya se felicitó el cumpleaños este año. Cero tokens, cero contraseñas.

Protección de rutas, roles y el botón "atrás"

Guard de página por rol — `estructura_base_controller.js`, función `iniciar(config)`:

```
const sesion = SesionService.verificarSesionSync();
if (!sesion) location.replace('../Cuentas/login.html');
if (!config.rolesPermitidos.includes(sesion.rol)) {
  location.replace('../acceso_denegado.html');
}
```

El rol nunca se decodifica de un JWT en el navegador — siempre se obtiene fresco desde `/api/auth/me/empleado`.

Bloqueo del botón "Atrás" (bfcache):

```
window.addEventListener("pageshow", (e) => {
  if (e.persisted) location.reload();
});
```

Cuando el navegador restaura una página desde el bfcache tras pulsar "Atrás", esto fuerza una recarga real que vuelve a ejecutar la verificación de sesión.

Interceptor global de fetch

`js/controllers/fetch_con_credenciales.js` reemplaza `window.fetch` una sola vez y agrega: `credentials: "include"` siempre; reintentos con backoff (1.5s, 3s, 5s, 8s) en errores 502/503/504 (cold start de Heroku); desenvoltura automática del `ApiResponse`; logout automático si una llamada responde 401/403 fuera de `/api/auth/`.

> También existe `bloqueo_menu_desarrollador.js` (bloquea F12, clic derecho). Es solo disuasión visual, no seguridad real — la seguridad real está en el backend.

Llamadas a la API, validaciones y Service Worker

`fetch` nativo en todos los services (sin axios). Base URL de Heroku escrita en cada `*_service.js`. Errores mostrados con SweetAlert2.

Validaciones del cliente: HTML5 nativo (`required`, `type="email"`, `type="date"`, `maxlength`) + validación manual en JS (longitud de contraseña, confirmación, verificación de duplicados antes de guardar).

Service Worker (`sw.js`): no cachea assets estáticos — su único trabajo es Web Push: recibe el evento `push`, muestra la notificación, y abre/enfoca la pestaña correcta en `notificationclick`.

WebSocket en el Frontend

`js/services/websocket_service.js`: conexión nativa hacia `wss://petcore-8afada45fabc.herokuapp.com/ws`. Heartbeat manual (ping cada 25s), reconexión con backoff exponencial + jitter (1s→30s), evento sintético `WS_RECONECTADO`. Se usa para la campanita de notificaciones, el indicador de conexión y actualizaciones en vivo de citas/asistencias.

La app móvil: Capacitor

Carpeta `PetCore-APP-Oficial/`. Capacitor toma el mismo frontend web y lo empaqueta dentro de un WebView nativo — `public/` contiene una copia exacta de `css/`, `js/`, `pages/` y `sw.js` del Frontend.

- `appId`: `com.petcore.app.oficial`, `webDir`: `dist` (carpeta que genera Vite).

- Plugin `CapacitorCookies` habilitado — imprescindible para que la cookie httpOnly funcione en el WebView nativo.
- Plugin `CapacitorHttp` habilitado — las peticiones pasan por el puente nativo.
- `SocialLogin` (Google habilitado) para login con Google desde la app.
- Dependencias: `@capacitor/core`, `/android`, `/ios`, `/cli`, `/push-notifications`, `/filesystem`, `/share`, `/status-bar`, `/app`.

Compilación para Android

PASO	COMANDO	QUÉ HACE
1	`npm run build`	Vite compila el frontend a `dist/`
2	`npx cap sync android`	Copia `dist/` al proyecto Android y sincroniza plugins
3	Android Studio / `gradlew`	Compila el APK/AAB (`minSdkVersion 24`, `targetSdk 36`)
4	Instalación física	Lo que pide la rúbrica: app instalada en un dispositivo Android físico

Organización del CSS y diseño responsive

Variables CSS en `css/style_global_compartido.css` bajo `:root[data-theme="dark"]`, alternadas cambiando un atributo en `<html>`. Media queries explícitas, detección de dispositivo móvil combinando `navigator.userAgent` + ancho de pantalla. Librerías visuales: Bootstrap 5.3.8, Animate.css y SweetAlert2.

SECCIÓN 4 · BASE DE DATOS · ORACLE CLOUD

Modelo de datos de PetCore

48 tablas en Oracle gestionadas con SQL Developer, estándar `SCREAMING_SNAKE_CASE`: `TBL_<ENTIDAD>`, constraints nombradas `PK_/FK_/UQ_/CK_/IX_`.

Estándar de nombres

ELEMENTO	CONVENCIÓN	EJEMPLO
Tabla	`TBL_<ENTIDAD>`	`TBL_EMPLEADOS`
Columna	`<PREFIJO>_<CAMPO>`	`EMP_CORREO`
Llave primaria	`PK_<CAMPO>`	`PK_EMP_ID`
Llave foránea	`FK_<ORIGEN>_<DESTINO>`	`FK_EMP_ROL`
Único	`UQ_<CAMPO>`	`UQ_EMP_CORREO`
Check	`CK_<CAMPO>`	`CK_EMP_DUI` (regex `00000000-0`)

TBL_EMPLEADOS: la tabla de usuarios del sistema

COLUMNA	TIPO / REGLA	PARA QUÉ
`EMP_CORREO`	UNIQUE + CHECK regex email	Usuario de login y avisos
`EMP_CONTRASENIA`	VARCHAR2(300)	Hash Argon2 — nunca texto plano
`EMP_DUI`	CHAR(10) + CHECK regex	DUI de El Salvador, formato `00000000-0`
`EMP_ROL_ID`	FK → TBL_ROLES	Rol del empleado
`EMP_TUR_ID`	FK → TBL_TURNOS (nullable)	Turno asignado
`EMP_ESTADO`	CHECK IN ('Activo','Inactivo','Suspendido')	Baja lógica

Las 48 tablas, por dominio funcional

Catálogos y personal

`TBL_ROLES` `TBL_TURNOS` `TBL_EMPLEADOS` `TBL_CFG_SISTEMA`

Catálogos clínicos y mascotas

`TBL_ESPECIE` `TBL_RAZA` `TBL_PELAJE` `TBL_PROPIETARIOS` `TBL_MASCOTAS` `TBL_ALERGIAS` `TBL_ALERGIA_MASCOTA` `TBL_DIAGNOSTICOS` `TBL_DIAGNOSTICO_RAZA` `TBL_MASCOTA_ENFERMEDAD`

Citas y atención clínica (el núcleo del sistema)

`TBL_SERVICIOS` `TBL_CITAS` `TBL_CITA_SERVICIO` `TBL_EXPEDIENTE` `TBL_DIAGNOSTICO_EXPEDIENTE` `TBL_EXPEDIENTE_SERVICIO` `TBL_PRESCRIPCION` `TBL_DETALLE_PRESCRIPCION` `TBL_HOSPITALIZACION`

`TBL_EVOLUCION_CLINICA` `TBL_TAREAS_MEDICAS` `TBL_CARTILLAS` `TBL_ANTIPARASITARIO` `TBL_HISTORIAL_TRATAMIENTO` `TBL_CUIDADOS` `TBL_POSOLOGIA_FARMACO`

Inventario, compras y ventas (farmacia)

`TBL_PROVEEDOR` `TBL_TIPO_PRODUCTO` `TBL_MARCA` `TBL_PRODUCTO` `TBL_COMPRAS` `TBL_DETALLE_COMPRA` `TBL_SOLICITUDES_BODEGA` `TBL_DETALLE_SOLICITUD` `TBL_VENTAS` `TBL_DETALLE_VENTA` `TBL_MOVIMIENTO_INVENTARIO`

Comunicación y notificaciones

`TBL_ANUNCIOS` `TBL_ANUNCIO_REACCIONES` `TBL_ANUNCIO_COMENTARIOS` `TBL_NOTIFICACIONES` `TBL_NOTIFICACION_DESTINATARIO` `TBL_SUSCRIPCIONES_PUSH` `TBL_PUSH_TOKEN_FCM`

Recursos humanos

`TBL_ASISTENCIA_EMPLEADOS` `TBL_DIAS_FERIADOS`

Relaciones clave: el flujo clínico completo

```
PROPIETARIO → MASCOTA → CITA → CITA_SERVICIO → EXPEDIENTE → DIAGNOSTICO_EXPEDIENTE → PRESCRIPCION
→ DETALLE_PRESCRIPCION → MOVIMIENTO_INVENTARIO
```

Un propietario tiene mascotas → la mascota agenda una cita (que puede incluir varios servicios, de ahí la tabla intermedia `CITA_SERVICIO`) → el veterinario abre/actualiza el expediente → registra un diagnóstico → si hace falta medicar, crea una prescripción con sus detalles → al despachar la receta en farmacia se genera un movimiento de inventario que descuenta el stock.

SECCIÓN 5 · ROLES, AUTENTICACIÓN Y CRUD

Roles, autenticación y funcionalidad CRUD

4 roles de personal, uno por módulo del negocio, más un 5º rol para el propietario de mascotas en la app móvil.

ROL	ENFOQUE	MÓDULOS PRINCIPALES
Administrador	Control total	Personal, inventario, compras, servicios, turnos, días feriados, asistencias, reportes, configuración
Veterinario	Atención clínica	Expedientes, citas, diagnósticos, prescripciones, hospitalización, cuidados, alergias, catálogos clínicos
Recepcionista	Front desk	Registro de propietarios/mascotas, agenda de citas, sala de espera
Farmacéutico	Farmacia / POS	Punto de venta, stock, posología, registro de ventas

Matriz de funcionalidad CRUD por rol

MÓDULO	ADMIN	VETERINARIO	RECEPCIONISTA	FARMACÉUTICO
Personal (empleados)	CRUD	—	—	—
Inventario / productos	CRUD	Leer	—	Leer / Stock
Compras a proveedor	CRUD	—	—	—
Servicios y turnos	CRUD	Leer	Leer	—
Propietarios y mascotas	Leer	Leer	CRUD	—
Citas	Gestionar	Atender / estado	Crear / agendar	—
Expedientes clínicos	—	CRUD	—	—
Diagnósticos y prescripciones	—	CRUD	—	Leer (para despachar)
Hospitalización	—	CRUD	—	—
Punto de venta (POS)	—	—	—	CRUD
Reportes	Generales	Clínicos	—	—
Configuración del sistema	Editar	Leer	Leer	Leer

Cómo demostrar "endpoints públicos vs. protegidos"

1. **Endpoint público sin login:** `GET /api/cfgs_sistema` responde 200 sin sesión — necesario para pintar el login.
2. **Endpoint protegido sin sesión → 401:** intentar `GET /api/empleados` sin cookie válida responde **401 Unauthorized**.
3. **Rol equivocado → 403 / acceso denegado:** logueado como Recepcionista, navegar a una URL de Administrador redirige a `acceso_denegado.html`; por API responde **403 Forbidden**.

4. **Botón "Atrás" tras cerrar sesión:** gracias al listener de `pageshow`, la página se recarga y vuelve a pedir login.

SECCIÓN 6 · LO QUE MÁS PREGUNTAN EN LA DEFENSA INTERNA

Conceptos clave, explicados con el código real

¿Qué es CORS y cómo está configurado en tu API?

CORS (Cross-Origin Resource Sharing) es el mecanismo del navegador que decide si un sitio web puede hacer peticiones a una API en otro dominio. Por defecto el navegador bloquea esas peticiones cruzadas, a menos que el servidor lo permita explícitamente. En PetCore, el frontend (Vercel) llama a la API (Heroku) — dominios distintos. La configuración vive en `CorsConfig.java`: lista blanca de orígenes, métodos permitidos y `allowCredentials=true` (obligatorio porque la sesión viaja en cookie). Frase clave: "CORS no es seguridad contra atacantes — es una política del navegador; la seguridad real de quién puede hacer qué la decide Spring Security en el backend".

¿Qué son los WebSockets y para qué los usas?

HTTP normal es pregunta-respuesta; un WebSocket abre un canal bidireccional persistente. PetCore implementa WebSocket puro (sin STOMP) en `WebSocketConfig.java` + `EventoWebSocketHandler.java`, endpoint único `/ws`, con broadcast total a todas las sesiones. Se usa para: notificaciones en tiempo real, indicador de conectado/desconectado, y actualizar citas/asistencias sin recargar.

¿Qué guardas en localStorage? ¿Por qué no el token?

`localStorage` es accesible desde cualquier JavaScript de la página, incluido código malicioso XSS. Por eso PetCore no guarda el token ahí — lo guarda el servidor en una cookie `httpOnly` (`authToken`), que JavaScript no puede leer ni robar. Solo se guardan preferencias de interfaz sin información sensible: tema, animaciones, formato de hora.

¿Cómo funciona tu JWT? ¿Por qué en cookie y no en header?

Un JWT es un texto firmado con 3 partes que contiene información del usuario y una firma que garantiza que nadie lo alteró. PetCore lo entrega dentro de una cookie `httpOnly` (en vez del típico header `Authorization: Bearer` + localStorage), combinando lo mejor de ambos mundos: stateless pero inaccesible a JavaScript. Implementa sesión deslizante: cada request renueva la cookie (15 min de vida).

Si usan cookies, ¿cómo evitan CSRF?

CSRF es un ataque donde un sitio malicioso hace que el navegador de la víctima envíe, sin darse cuenta, una petición a otra app donde tiene sesión activa. PetCore mitiga con la cookie configurada `SameSite` + `Secure`, CORS limitando qué orígenes reciben respuesta, y al ser una API JSON pura se reduce la superficie de ataque clásica. Por eso se deshabilita la protección CSRF integrada de Spring Security (pensada para formularios) y se usa `SameSite` + CORS + JWT.

¿Cómo bloquean el botón "Atrás" después de cerrar sesión?

El navegador puede restaurar una página desde el bfcache (back-forward cache) sin recargarla. PetCore escucha el evento `pageshow`: si `event.persisted` es true, fuerza un `reload()`, que vuelve a ejecutar el guard de sesión y redirige al login si ya no hay cookie válida.

¿Cómo protegen tantos endpoints sin equivocarse?

Patrón "deny by default": cada endpoint necesita una regla explícita, y la última línea de la configuración es `auth.anyRequest().denyAll()`. Un endpoint nuevo sin regla queda bloqueado automáticamente, en vez de quedar expuesto por accidente.

¿Cómo funciona el login con Google?

Para propietarios en la app móvil. El plugin `SocialLogin` de Capacitor abre el selector de Google, que devuelve un ID Token firmado. `GoogleTokenVerifier.java` usa `google-api-client` para verificar la firma directamente con los servidores de Google (nunca se confía ciegamente en el cliente). Si es válido, se crea/vincula el `PropietariosEntity` con `pro_google_id` y se emite la cookie `authTokenPropietario`.

¿Es PetCore una PWA? ¿Qué hace el Service Worker?

Parcialmente. `sw.js` no cachea assets estáticos; su única función es recibir y mostrar notificaciones push del navegador (payload firmado con claves VAPID vía `web-push`). El push nativo de la app móvil usa Firebase Cloud Messaging — son dos canales separados para dos plataformas distintas.

¿Dónde se valida realmente un dato: frontend o backend?

En ambos, con propósitos distintos. El frontend valida para dar feedback inmediato, pero se puede saltar (DevTools, Postman). La validación que garantiza integridad ocurre en el backend con Bean Validation + `GlobalExceptionHandler`. Regla de oro: nunca confiar en que un dato ya viene validado solo por haber pasado por el frontend.

¿Por qué no hicieron la app móvil "nativa" desde cero?

Capacitor toma la app web (mismo HTML/CSS/JS) y la empaqueta en un WebView nativo con acceso a APIs del dispositivo vía plugins. Ventaja: un solo código fuente para web y móvil. Lo 100% nativo son los plugins: push, filesystem, share, status bar.

SECCIÓN 7 · CRITERIO 10 · DOMINIO TEÓRICO Y FUNCIONAL

Banco de preguntas para la defensa externa

Organizado por criterio de la rúbrica. El jurado valora seguridad y lenguaje técnico más que una respuesta perfecta de memoria.

1. Documentación del sistema

¿Qué documentación tienen y para quién es cada una?

Manual de usuario (con capturas paso a paso, para el personal de la clínica) y documentación técnica (instalación/ejecución local) más diagramas (casos de uso, entidad-relación) que explican el funcionamiento interno.

¿Dónde está el diagrama de casos de uso y qué muestra?

Debe mostrar, por cada rol, qué acciones puede realizar — base del criterio 8: cada caso de uso corresponde a una regla real en `SecurityConfig.java`.

2-3. Diseño de interfaces (web y móvil)

¿Cómo logran que las interfaces sean responsive?

Bootstrap 5.3.8 (grid, utilidades flex) + media queries propias + detección de dispositivo móvil vía JavaScript para decidir sidebar fija vs. menú hamburguesa.

¿Por qué un diseño distinto por rol?

Cada rol tiene necesidades de trabajo distintas; `pages/` está organizado por rol con su propia sidebar y flujo, mejorando usabilidad real frente a un dashboard único sobrecargado.

4-5. Funcionalidad del sistema (CRUD)

Demuestre un CRUD completo end-to-end

Crear → Leer (listado paginado actualizado) → Actualizar → Eliminar. Mencionar que cada acción dispara un `fetch` y la respuesta viene envuelta en `ApiResponse`.

¿Qué pasa si intentan eliminar un registro en uso?

Las FK lo impiden a nivel de integridad referencial; el backend captura `DataIntegrityViolationException` y responde 409 con mensaje claro, no un error 500.

6. Validaciones e integridad de datos

¿Qué pasa si envío un correo mal formado o fecha futura de nacimiento?

`@Email` y `@Past` disparan `MethodArgumentNotValidException` → 400 con el campo específico, antes de ejecutar lógica de negocio.

¿Validan formato de DUI? ¿Cómo?

En dos capas: CHECK `REGEXP_LIKE` en la base de datos y `@Pattern` en el DTO del backend.

7. Autenticación y roles de usuario

¿Qué pasa exactamente cuando hago login? Paso a paso.

1) Frontend manda credenciales. 2) Backend compara hash Argon2. 3) `JWTUtils` firma el JWT y lo pone en cookie httpOnly. 4) Navegador guarda la cookie. 5) Frontend redirige al dashboard del rol. 6) Cada petición futura pasa por `JwtCookieAuthFilter`.

¿Cómo sabe el frontend qué rol tiene el usuario logueado?

Nunca decodifica el JWT en el navegador; llama a `GET /api/auth/me/empleado`, que responde con la identidad validada por el servidor.

8. Protección de endpoints y vistas

Si copio el link de una página de Administrador siendo Recepcionista, ¿qué pasa?

El guard compara el rol real contra `rolesPermitidos` y redirige a `acceso_denegado.html`. Aunque se desactive JavaScript, la API sigue respondiendo 403, porque la regla vive también en `SecurityConfig.java`.

¿Qué endpoints son públicos y por qué?

Solo los necesarios antes de tener sesión: configuración del sistema, login/logout/recuperación, clave pública VAPID, y "crear primer administrador" (solo funciona si no existe ningún empleado).

9. Despliegue

¿Por qué Heroku para el backend y Vercel para el frontend?

El frontend es estático (Vercel, CDN global gratis); el backend es una app Java con estado de ejecución (Heroku, hosting de "dyno"). Refleja una arquitectura realista: frontend y backend se despliegan y escalan de forma independiente.

¿Qué significa "configurada para recibir peticiones de orígenes definidos"?

Es la configuración de CORS: la API solo acepta peticiones de la lista blanca definida en `CorsConfig.java`.

10. Dominio teórico y funcional

¿Cuál fue el mayor reto técnico del proyecto?

Coordinar el modelo "deny by default" a través de ~47 módulos sin dejar ningún endpoint sin proteger, y sincronizar manualmente la lista de orígenes de CORS entre HTTP y el handshake de WebSocket.

Si tuvieran más tiempo, ¿qué mejorarían?

Securizar el handshake de `/ws` (hoy público por diseño) y añadir cacheo de assets estáticos en el Service Worker para mejor soporte offline.

¿Por qué Oracle y no MySQL/PostgreSQL?

Requisito de la institución, gestionado con SQL Developer en Oracle Cloud. El driver `ojdbc11` + Hibernate está configurado con el dialecto de Oracle; `GenerationType.IDENTITY` elegido por compatibilidad con el autoincremento de Oracle.

SECCIÓN 8 · PARA PRESENTAR AL JURADO

Guion de presentación de PetCore

Estructura sugerida siguiendo el orden del documento de defensa externa: web → API → base de datos → app móvil → dominio teórico.

1. Apertura

0:00 – 1:00

"Somos el equipo PetCore, y desarrollamos un sistema integral de gestión veterinaria pensado para una clínica real: desde que un propietario agenda una cita hasta que su mascota recibe tratamiento y se despacha la receta en farmacia. El sistema tiene cuatro roles de personal — Administrador, Veterinario, Recepcionista y Farmacéutico — más una app móvil para que los propietarios gestionen a sus mascotas."

Quién habla: el líder del equipo. Tono seguro, sin leer.

2. Demo del sistema web (Vercel)

1:00 – 6:00

Login en vivo (URL pública, no localhost) → dashboard Administrador → CRUD completo → cambio de rol mostrando un dashboard distinto. Demostrar protección de endpoints (intentar entrar a otra URL de rol → acceso denegado) y el bloqueo del botón "Atrás" tras cerrar sesión.

3. Demo de la API (Heroku)

6:00 – 9:00

Llamar un endpoint público (200 OK) y el mismo protegido sin cookie (401) en Postman/consola. Explicar el envoltorio `ApiResponse`. Mencionar CORS: "estas peticiones solo funcionan desde los orígenes que configuramos explícitamente".

4. Demo de la base de datos (Oracle Cloud)

9:00 – 12:00

SQL Developer conectado a Oracle Cloud. Mostrar el modelo de tablas, una consulta que refleje el dato recién creado en la web, y un ejemplo de constraint explicando su propósito.

5. Demo de la app móvil (Android físico)

12:00 – 16:00

Login como propietario (incluyendo Google si aplica), mostrar mascotas, agendar/ver una cita, y una notificación push si es posible. Mencionar que es el mismo frontend web empaquetado con Capacitor.

6. Arquitectura y dominio técnico

16:00 – 19:00

Resumen de capas del backend, flujo de autenticación (JWT en cookie httpOnly), patrón "deny by default". Aquí suele empezar el interrogatorio del jurado.

7. Cierre

19:00 – 20:00

"Con esto cubrimos el sistema web, la API, la base de datos y la app móvil de PetCore, todos desplegados en producción y funcionando de forma integrada. Quedamos atentos a sus preguntas."

Reparto sugerido de roles en la exposición

INTEGRANTE	RESPONSABILIDAD
Persona 1	Apertura + demo web (Administrador)
Persona 2	Demo web (Veterinario) + protección de endpoints
Persona 3	Demo API + base de datos
Persona 4	Demo app móvil + cierre

Ajustar según integrantes del equipo — lo importante es que **todos** puedan responder preguntas de cualquier parte, no solo la suya (el criterio 10 evalúa a cada integrante).